

CS253 Assignment 1 — Memory Efficient Versioned File Indexer

Jashan Mittal

240488

Problem Statement

Large text files are processed in chunks to avoid loading the entire file into memory at once. This assignment required building a word level indexer in C++ that reads a large text file using a fixed size buffer and answers queries about word frequencies.

A word level index means the program stores each unique word as a key and how many times it appears as its value. For example, if the file contains "error debug error info" then the index stores error as 2, debug as 1, and info as 1. This index is then used to answer the queries.

The buffer size must be between 256 KB and 1024 KB and must stay constant throughout execution. The file must never be fully loaded into memory at any point.

Queries Supported

The program supports three types of queries.

The **word** query takes a single file and a word and returns how many times that word appears in the file.

The **top K** query takes a single file and a number K and returns the K most frequent words in descending order of frequency.

The **diff** query takes two files and a word and returns the frequency of that word in each file along with the difference between them.

Approach to Solve the Problem

The main challenge was reading a large file without putting it all in memory at once. The solution is to read the file in small fixed size pieces. Each piece is processed and then the same memory is reused for the next piece. This way only a tiny part of the file is in memory at any time.

To get words out of each piece we go through it character by character. If the character is a letter or number we add it to the word being built. If it is anything else like a space or comma the word is done and we save it. Everything is converted to lowercase so Error and error count as the same word.

One problem with this approach is that a word can get cut in half between two buffers. To handle this we carry any unfinished word forward to the next buffer so it gets completed correctly.

Every word that comes out goes into a map that tracks how many times each word has appeared. This map is the index and it is used to answer all three queries. Word query is a simple lookup. Top K

sorts by count and takes the first K. Diff query builds two separate maps from two files and compares the counts.

Design and Classes

The solution uses eight classes in total. Four are the core required classes and the rest form the query hierarchy.

BufferedReader is responsible for opening the file and reading it one fixed size chunk at a time. The buffer is allocated once in the constructor and the same memory is reused on every read. This is what keeps memory usage constant.

Tokenizer is responsible for taking the raw bytes from a buffer and extracting clean lowercase alphanumeric words. The most important part of this class is handling words that are split across two buffers. For example if one buffer ends with the letters "segfau" and the next buffer starts with "It" the tokenizer must correctly produce the word "segfault". This is solved using a leftover string that carries the unfinished word from one chunk to the next.

VersionIndex is responsible for storing the word to frequency map for one named version of a file. It uses an unordered map(uses hash map) internally. Memory only grows with the number of unique words not with the size of the file.

QueryProcessor is responsible for running the full pipeline. It creates the BufferedReader, gives each chunk to the Tokenizer, inserts words into the VersionIndex, and then creates and runs the correct query.

Query is an abstract base class with two pure virtual methods called `execute` and `printResult`. **WordQuery**, **TopKQuery**, and **DiffQuery** all inherit from Query and each provides its own implementation of these methods.

C++ Features Used

Inheritance and abstract base class. Query is declared as an abstract base class using pure virtual functions. WordQuery, TopKQuery, and DiffQuery inherit from it and each overrides both virtual methods.

Runtime polymorphism. In QueryProcessor a base class pointer of type `unique_ptr<Query>` holds whichever derived object is needed. When `execute` is called through this pointer C++ looks up the correct function at runtime. The compiler does not know at compile time which `execute` will run because it depends entirely on what query type the user passed on the command line.

Function overloading. The `normalize` function in Tokenizer is overloaded twice. One version accepts a full string and is used to clean up the word typed by the user on the command line. The other version accepts a single character and is used inside the per byte scan loop for efficiency. The `getFrequency` function in VersionIndex is also overloaded. One version returns zero for missing words and the other lets the caller supply a custom default value.

Function template. The `findTopK` function is a generic template that works with any unordered map. It copies the map entries into a vector and uses partial sort to find the top K entries in descending order. Using partial sort means only the K positions we actually need are fully sorted which is faster than sorting the entire map ($O(k \log n)$ instead of $O(n \log n)$)

Exception handling. All runtime errors use throw, try, and catch. Invalid buffer size, missing files, bad integer arguments, and unknown query types all throw exceptions that bubble up to the catch block in main.

Assumptions and Observations

Word matching is case insensitive. Error, ERROR, and error are all counted as the same word.

If a word is not found in the file the program returns a frequency of zero rather than throwing an error.

For the diff query, if the first version has higher count for a word instead of returning a negative number it returns a positive number with a clear statement “more in v1”.

The buffer size has no effect on the final result. Whether the buffer is 256 KB or 1024 KB the word counts will always be the same. A larger buffer just means fewer reads and slightly faster execution.